

Issues, Findings, and Lessons Learned

ChatGPT / Codex Windows recovery, preservation, verification, and restoration

Prepared July 29, 2026

Scope. This document separates the failures, risks, misleading signals, and recovery findings from the procedural steps. It is intended as a technical incident summary and future troubleshooting reference.

1. Primary app failure

- The Microsoft Store ChatGPT/Codex Windows package repeatedly failed with AppModel Runtime event 208 and error 0x8000001A.
- The visible dialog stated: “The application is exiting and cannot service this request.”
- The app also entered update/repair loops, including error 0x80073D02 when package files were in use.
- The failed package was OpenAI.Codex version 26.721.4979.0.
- Resets, reinstalls, profile resets, AppContainer resets, Store/AppX repair, lifecycle attempts, DISM, SFC, and reboots did not permanently correct that build.

2. Excessive recovery cost

- The incident consumed nearly 48 hours from the last response near the crash on Monday at 11:53 PM to restoration late Wednesday night.
- The recovery required preservation of more than 262,000 files, two independent verified backup copies, a data export, Windows integrity scans, a replacement app build, conversation verification, and a custom offline sidebar merge.
- The app exposed poor recovery ergonomics: internal state, visible sidebar indexing, cloud-backed conversations, project roots, and the actual filesystem workspace behaved as separate recovery layers.

3. WindowsApps permission risk

- A broad ownership/permission command was accidentally started against C:\Program Files\WindowsApps.
- The WindowsApps root owner temporarily became Everyone. The root was repaired to NT SERVICE\TrustedInstaller.
- A recursive ownership operation reported 0 successful files and approximately 89,000 failures, which strongly suggested child ownership was not broadly changed.
- The correct lesson is to avoid /T, /R, or blanket ACL resets on the entire WindowsApps tree. Repair only the specific object that was changed.

4. Orphaned package residue

- An old package directory could remain under WindowsApps even after uninstall. Such residue is not, by itself, evidence of disk or Windows corruption.

C:\Program Files\WindowsApps\OpenAI.Codex_26.721.4979.0_x64__2p2nqsd0c76g0

- The folder resisted deletion. Broad WindowsApps permission changes would have been riskier than leaving the orphaned app residue in place.

5. Integrity-check interpretation

- DISM CheckHealth and ScanHealth both reported no component-store corruption.
- CHKDSK completed in 2.47 hours and reported no file-system problems and 0 KB in bad sectors.
- SFC initially repaired protected files, but the recurring repair was a OneDrive shortcut in the Default user profile, not broad system damage.

C:\Users\Default\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\OneDrive.lnk

- A later SFC scan reported no integrity violations. There is no evidence that ChatGPT/Codex caused the SFC repair; only temporal association was observed.

6. Time and ETA errors during troubleshooting

- CHKDSK percentage and ETA values were volatile and should not have been treated as reliable completion predictions.
- Several earlier responses assigned exact timestamps without a grounded time source. Future reports should use only console timestamps, log timestamps, or an actual time tool.
- Progress should be judged by changing counters and log activity, not by unstable ETAs.

7. Store metadata was insufficient

- Winget showed Store product ID 9PLM9XGG6VKS and refreshed product text but reported Version: Unknown.
- Listing metadata changes did not prove that a new executable build existed.
- The decisive check was Get-AppxPackage after installation. It showed replacement build 26.721.11231.0, which was newer than the failed 26.721.4979.0 build.

8. Local .codex data was misleading

- The old .codex directory contained sizeable SQLite WAL files, which initially suggested recoverable task history might be present.
- Read-only schema inspection showed the key tables were empty: threads = 0, goals = 0, memories = 0.
- The logs database contained 228 rows and remains useful for later diagnosis, but it could not restore chats or projects.
- The old config.toml contained only a trust entry for C:\Windows\System32, likely inherited from an elevated shell working directory.
- The old skills and plugins trees contained only system-managed or cached content. No personal portable data was found there.

- Copying the complete old .codex directory over the new installation would have risked credentials, installation identifiers, caches, runtime paths, and version-sensitive database state.

9. Conversation recovery model

- Codex tasks/chats were cloud-backed and visible through the recovered account/project state, even though the old local threads table was empty.
- The export was a preservation and comparison source, not a direct import package.
- All 148 exported conversations were accessible in Codex, including all 3 archived conversations. One additional newer conversation existed after the export cutoff.
- Database replacement was therefore unnecessary and inappropriate for conversation recovery.

10. Sidebar and project-index separation

- The underlying conversations and workspace were recovered before the sidebar visibly reflected them.
- The sidebar was a separate local UI/index problem, not proof that the chats or projects were missing.
- A dry-run-tested offline merge restored 261 thread index entries, 24 workspace roots, 22 project-order entries, and 10 pinned threads while preserving current recovery tasks.
- The restore script created a rollback snapshot before modifying the local index.

11. Backup performance and path-depth issues

- Copy-Item failed on deeply nested temporary plugin paths and later appeared to hang while walking generated dependency trees.
- The problematic paths were under .codex\.tmp and generated plugin structures. These were not necessary portable recovery data.
- Robocopy with explicit exclusions and process/log monitoring was more reliable for the large project tree.
- Do not restart a long-running backup merely because the command window times out. First check whether the process is active and whether the log is advancing.

12. What worked well

- Two independent, verified preservation copies before destructive changes.
- A separate account data export extracted under C:\CodeProjects\Export.
- Windows integrity checks performed before reinstalling the app.
- Verifying the actual installed AppX version rather than trusting Store metadata.
- Read-only database inspection instead of immediate transplantation.
- Comparing all exported conversations against live Codex visibility.
- Creating a post-recovery project snapshot after conversation verification.
- Dry-running the sidebar merge and creating rollback data before the real change.

13. Current evidence retained

Location	Purpose
C:\Users\Owner\Desktop\ChatGPT-Codex-Preservation-20260728-101536	Primary preservation set
C:\backupCodeProjects\ChatGPT-Codex-Preservation-20260728-101536	Second verified preservation copy
C:\CodeProjects\Export	Export, recovery inventory, logs, and restore package
C:\Users\Owner\Desktop\codex-old-db-analysis	Read-only old database analysis copy
C:\Users\Owner\Desktop\CodeProjects-post-recovery-20260729	Post-recovery workspace snapshot

14. Final assessment

The replacement build is functional, the workspace is preserved, all exported Codex conversations are verified, and the sidebar/project index has been restored. The incident demonstrated that the Windows app's lifecycle and recovery behavior were the principal failure points, while Codex's account-backed conversation data and recovery capabilities ultimately enabled a complete restoration.